In [ ]:
```
import pandas as pd
import numpy as np
import altair as alt
import matplotlib as plt
import polars as pl
import folium
import marimo as mo
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt
RANDOM_STATE = 42
```

🚗 Un acercamiento al analisis de datos de transito usando algoritmos de aglomeración 🚗

> Autor: Jesus Ivan Rivera Ramirez Materia: Fundamentos de IA

Problema ¿Se pueden caracterizar las regiones de con mayor numero de incidentes de transito?

El acercamiento:

Limpieza de datos -> DBSCAN -> K-Means -> Caracterización

Dataset

Dataset: Hechos de transito

El dataset utilizado corresponde a hechos de transito,

- **Hechos de transito**: contiene información sobre los hechos de transito ocurridos en la cdmx recolactados por las alcaldias, ssc y C5.

Limpieza y preporcesamiento

Para los hechos de transito tomaremos en cuenta las columnas de:

- hora_evento 🕛
- tipo_evento 💥
- latitud 📍
- longitud 📍
- dia 🕛
- tipo_de_interseccion 🏘️

Para la limpieza de estas columnas comenzaremos filtrando las alcaldias para eliminar datos más registrados eliminar los datos mal registrados de las alcaldias no supone una defirencia notable pues estos solo contenian un registro.

Filtraremos los días de la semana pues estos los usaremos más adelante en el analisis.

Tambien delimitaremos los hechos que en realidad ocurrieron dentro de la cdmx pues el analisis exploratorio mostro que habia datos registrados fuera de la ciudad.

In [ ]:
```
hechosDf = pd.read_csv('data/transito.csv')

hechosDf = hechosDf[hechosDf.alcaldia.isin(['BENITO JUAREZ','COYOACAN','GUSTAVO A
MADERO',
 'TLAHUAC',  'MIGUEL HIDALGO',  'CUAUHTEMOC',
   'IZTACALCO', 'VENUSTIANO CARRANZA',  'CUAJIMALPA',
 'TLALPAN',  'IZTAPALAPA',  'MILPA ALTA',
'AZCAPOTZALCO',  'XOCHIMILCO', 'MAGDALENA CONTRERAS',
  'ALVARO OBREGON'])]
hechosDf = hechosDf[hechosDf.dia.isin(['Lunes','Martes','Miércoles', 'Jueves',
'Viernes', 'Sábado', 'Domingo'])]
hechosDf = hechosDf[(hechosDf.latitud<= 19.6) & (hechosDf.latitud>=19.1) &
(hechosDf.longitud<=-98.9)&(hechosDf.longitud>=-99.4)]

columnas_hechos =
['hora_evento','tipo_evento','latitud','longitud','dia','tipo_de_interseccion']
hechosDf = hechosDf[columnas_hechos].reset_index(drop=True)
```

Vemos que la columna hora_evento tiene varios valores nulos, la estrategia que se usara sera eliminarlos pues no hay forma obvia de llenarlos, por ejemplo con el promedio o volver a tomar los datos.

In [ ]:
```
hechosDf.isna().sum()
```

despues de eliminar los nulos tenemos nuestro dataset listo para trabajar

In [ ]:
```
df = hechosDf.dropna()
```

In [ ]:
```
df.isna().sum()
```

Exploración

Veamos como se ve la distribución de puntos sobre un mapa de la Ciudad de Mexico.

In [ ]:
```
_plot_points = df.sample(min(5000,len(df)), random_state=RANDOM_STATE)
[["latitud","longitud"]]
_center = [_plot_points["latitud"].mean(),_plot_points["longitud"].mean()]
_map = folium.Map(location=_center, tiles='OpenStreetMap', prefer_canvas=True,
zoom_start=12)
for _lat, _lon in _plot_points.itertuples(index= False, name=None):
```

```
    folium.CircleMarker(location=[_lat,_lon], radius=3, fill=True).add_to(_map)
_map
```

Podemos observar que los datos estan concentrados en la región centro de la ciudad de mexico, veamos la distribución de la latitud y longitud

Distribución de la latitud

Distribución de la longitud

Las graficas confirman lo percibido por la distribución de puntos sobre el mapa, la región centro es la que mayor densidad de puntos tiene. Tambien se puede observar que en ambas distribuciones hay varios puntos anomalos. Estos puntos pueden corresponder a incidentes en la periferia de la ciudad. Debido a la los puntos extremos y concentración de los incidentes en cierto intervalo se usara **DBSCAN** pues nos ayudara a encontrar concentraciones mas locales.

Puntos de mayor densidad usando DBSCAN

Consideraciones generales, como los puntos estan representados sobre la tierra, es natural tomar la distancia de **haversine**, esta nos ayudara a medir correctamente la distancia entre dos puntos sin necesidad de normalizar/estandarizar.

Al explorar la los parametros de DBSCAN se observo que usar distancias muy grandes $\approx 300m$ genera un grupo que domina sobre los demas. Al estar interesados en zonas más locales buscaremos una configuración de parametros que nos de clusters más compactos y con una cantidad similar de puntos.

In [ ]:
```
EARTH_RADIUS = 6371.0088
coords = np.radians(df[["latitud","longitud"]].to_numpy())
results = []
for eps in range(50,120,10):
    eps = eps/1000
    for minS in range(10,35,5):
        _dbscan = DBSCAN(
        eps=eps / EARTH_RADIUS,
        min_samples=minS,
        metric='haversine',
        algorithm='ball_tree',
        n_jobs=-1
        )
        _labels = _dbscan.fit_predict(coords)
        _tmp = df.copy()
        _tmp['cluster'] = _labels
```

```
        noise = (_labels == -1).mean() * 100
        clustered = _tmp[_tmp['cluster'] >= 0]
        sizes = clustered.groupby('cluster').size()
        n_cluster = len(sizes)
        extension = []
        for _, group in clustered.groupby('cluster'):
            lat = group['latitud']
            lon = group['longitud']
            dy = (lat.max() - lat.min()) * 111320
            dx = (lon.max() - lon.min()) * 111320 * np.cos(np.radians(lat.mean()))
            extension.append(np.hypot(dx, dy))

        results.append({
        "eps": eps,
        'min_samples': minS,
        'clusters': n_cluster,
        'Ruido': noise,
        'tamaño_minimo': sizes.min() if len(clustered) else np.nan,
        'tamaño_media': sizes.median().round(3) if len(clustered) > 0 else np.nan,
        'cluster_mas_grande': round(sizes.max() / sizes.sum(), 3) * 100 if
len(clustered) > 0 else np.nan,
        'extension_media': np.median(extension).round(3) if extension else np.nan,
        'extension_p90': np.percentile(extension, 90).round(3) if extension else
np.nan
        })

summary = pd.DataFrame(results)
summary['dif'] = summary.extension_p90 - summary.extension_media
summary
```

Buscaremos los cambinación de parametros que no tengan un cluster dominante, cosideren gran parte de los datos y tengan una extensión pequeña.

In [ ]:
```
summary[summary.cluster_mas_grande<10]
```

Los parametros elegidos son $eps = 90m$ y $minSamples = 20$, es el que mejor balancea

Dato curioso, minSamples no determina el numero minimo de puntos que tiene un cluster.

In [ ]:
```
dbscan = DBSCAN(
eps=.09/ EARTH_RADIUS,
min_samples=20,
metric='haversine',
algorithm='ball_tree',
n_jobs=-1
)
labels = dbscan.fit_predict(coords)
puntos = df.copy()
puntos['cluster'] = labels
puntosSR = puntos[puntos.cluster>=0]
```

Cluster encontrados por DBSCAN

- clusters: 1310
- ruido: 39.9

In [ ]:
```
showMap(puntos)
```

Como nos interesa agrupar los puntos de cada cluster analicemos la distribución de los tamaños

In [ ]:
```
_sizes = pd.Series(labels).loc[lambda x: x!=-1].value_counts()
#print(sizes)
print(_sizes.describe(percentiles=[0.1,0.25,0.5,0.75,0.9]))
plt.boxplot(_sizes, orientation='horizontal')
```

K-medias

Preprocesamiento

Ahora consideraremos cada uno de estos puntos como un cluster y consideremos las siguientes caracteristicas

- porcentaje de accidentes de noche (despues de las 8 antes de las 6)
- porcentaje de accientes en horas pico (entre las 7 y las 10 o entre las 5 y las 8)
- porcentaje de accidentes en fin de semana
- porcentaje de accidentes clasificados como choque
- porcentaje de accidentes clasificados como atropello
- porcentaje de accidentes clasificados como otros

consideraremos solamnete los clusters que tienen mas de 20pts, esto nos ayudara a la estabilidad numerica.

In [ ]:
```
features = ['noche','pico','fin','choque', 'peaton', 'otro']
tmp = puntosSR.copy()
tmp['hora'] = pd.to_datetime(tmp['hora_evento'], format='%H:%M:%S').dt.hour
tmp['noche'] = ((tmp['hora']>20) | (tmp['hora']<=6)).astype('Float64')
tmp['pico'] = (((tmp['hora']>=7) & (tmp['hora']<=10)) | ((tmp['hora']>=17) &
(tmp['hora']<=20))).astype('Float64')
tmp['fin'] = (tmp['dia'].isin(['Sábado', 'Domingo'])).astype('Float64')
tmp['choque'] = (tmp['tipo_evento'].isin(['CHOQUE'])).astype('Float64')
tmp['peaton'] = (tmp['tipo_evento'].isin(['ATROPELLADO'])).astype('Float64')
tmp['otro'] = (~tmp['tipo_evento'].isin(['ATROPELLADO',
'CHOQUE'])).astype('Float64')
kdf = tmp.groupby('cluster', as_index=False)[features].mean()
kdf['hechos'] = tmp.groupby('cluster').size()
kdf = kdf.reset_index(drop=True)
kdf = kdf[(kdf['hechos']>=20)]
kdf
```

Veamos la distribución de estos valores

Podemos observar que las variables definidas tienen variabilidad tanto en las caracterisiticas temporales como en las de incidentes, por lo que resulta apropiado para aplicar k medias.

Ahora analicemos la relación que existe entre las variables.

In [ ]:
```
kdf[features].corr().style.background_gradient(cmap='viridis')
```

Podemos observar que hay 5 correlaciones importantes

- noche con pico
- noche con fin
- choque con peaton
- choque con otro
- peaton con otro

La correlación entre los accidentes es esperable pues estas variables se construyen de forma similar y son excluyentes entre sí, De forma similar ocurre con noche y pico, pues estos horarios son excluyentes. La interesante en noche con fin sugiere que los accidentes de noche ocurren más en fin de semana.

ahora procedemos a aplicar k medias, para ello primero escalaremos nuestos datos pues al tener diferente varianza, las variables con mayor varianza pueden influir más sobre los clusters.

Aplicando el metodo del codo para determinar el numero de cluster.

In [ ]:
```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
intertia = []
normalizer = StandardScaler()
X = normalizer.fit_transform(kdf[features])
for _i in range(2,7):
    _model = KMeans(n_clusters=_i,random_state=RANDOM_STATE)
    _labels1 = _model.fit_predict(X)
    intertia.append(_model.inertia_)
plt.plot(range(2,7), intertia)
```

El codo es claro en 3, y podemos corroborar esta decisión con la silueta.

In [ ]:
```
from sklearn.metrics import silhouette_score
silhouette = []
for _i in range(2,7):
    _model = KMeans(n_clusters=_i,random_state=RANDOM_STATE)
    _labels1 = _model.fit_predict(X)
    silhouette.append(silhouette_score(X, _labels1))
plt.plot(range(2,7), silhouette)
```

Veamos el tamaño de los clusters

In [ ]:
```
model = KMeans(n_clusters=3,random_state=RANDOM_STATE)
Klabels = model.fit_predict(X)
final = kdf.copy()
final['Klabels'] = Klabels

pd.Series(Klabels).value_counts()
```

Vemos que los clusters tienen tamaño similares, por lo que k medias no concentro la mayoria de observaciones en un solo grupo.

Analizando los centroides

In [ ]:
```
final.groupby('Klabels')
[features].mean().style.background_gradient(cmap='viridis')
```

Podemos clasificar los clusters como

- 0: Incidentes peatonales, este cluster presenta la mayor cantidad de incidentes peatonales.
- 1: Choques en hora pico, tiene la mayor cantidad de choques y a diferencia del cluster 2 ocurren con mayor frecuencia en hora pico
- 2: Nocturnos y de fin de semana: es el cluster con mayor cantidad de incidentes de noche y en fin de semana

Veamos como se ven el mapa

In [ ]:
```
showMap(viz,'kcluster')
```

Podemos repetir el experimento para los clusters quitando los puntos extremos.

In [ ]:
```
kdf1 = kdf.copy()
kdf1 = kdf1[(kdf1['hechos']>=20) & (kdf1['hechos']<=117)]
X1 = normalizer.fit_transform(kdf1[features])
_inertia = []
for _i in range(2,7):
    _model = KMeans(n_clusters=_i,random_state=RANDOM_STATE)
    _labels1 = _model.fit_predict(X1)
    _inertia.append(_model.inertia_)
plt.plot(range(2,7), intertia)
```

In [ ]:
```
_silhouette = []
for _i in range(2,7):
    _model = KMeans(n_clusters=_i,random_state=RANDOM_STATE)
    _labels1 = _model.fit_predict(X1)
    _silhouette.append(silhouette_score(X1, _labels1))
plt.plot(range(2,7), _silhouette)
```

In [ ]:
```
model1 = KMeans(n_clusters=3,random_state=RANDOM_STATE)
Klabels1 = model1.fit_predict(X1)
final1 = kdf1.copy()
final1['Klabels'] = Klabels1

pd.Series(Klabels1).value_counts()
```

In [ ]:
```
final1.groupby('Klabels')
[features].mean().style.background_gradient(cmap='viridis')
```

Podemos clasificar los clusters como

- 2: Incidentes peatonales, este cluster presenta la mayor cantidad de incidentes peatonales.
- 0: Choques en hora pico, tiene la mayor cantidad de choques y a diferencia del cluster 2 ocurren con mayor frecuencia en hora pico
- 1: Nocturnos y de fin de semana: es el cluster con mayor cantidad de incidentes de noche y en fin de semana

Que son similares a los encontrados sin quitar los puntos extremos.

Apendice

El analisis visual sugiere diferencias en la distribución del tipo de vialidad con respecto a los k clusters, analicemos rapidamente esto.

In [ ]:
```
_tabal = pd.crosstab(viz['kcluster'],
                     viz['tipo_de_interseccion'],
                     normalize= 'index').mul(100).round(3)
_tabal.style.background_gradient(cmap='viridis')
```

Podemos ver que para el cluster 2 predominan los accidentes sobre el resto de intersecciones, no solo en la cruz como para el cluster 0 y 1. Para el cluster 1 vemos que la mayoria de los accidentes suceden en la cruz y la T.

Esto sugiere que la caracterización de los clusters tambien presenta diferencias en las caracteristicas geometricas de las vialidades, aun cuando estas no fueron utilizadas para construir los clsuters.

In [ ]:
```

```